UCS503P – Project Report

ResearchAgent: An Autonomous AI Agent for Stock and Company Research

Prototype-Stage Report

Submitted by

Ananya Khanduja, CSED 1024030693
Garv Sharma, CSED 1024030695
Vishwa Choudhary, CSED 1024030668

Submitted to

Paramveer Ma'am

Mid-Semester Evaluation
Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

October 2026

Abstract

Researching a listed company commonly requires users to move between financial-data portals, news sources, and search engines before assembling a coherent view. This fragmentation increases effort and makes the resulting notes inconsistent. ResearchAgent is a course project that proposes a web-based, tool-using AI agent to accept a company name or ticker, gather available evidence through external tools, and synthesise it into a structured research brief.

The target architecture separates a React, Vite, and TypeScript frontend from a stateless Go HTTP API. Its proposed agent loop follows a Plan–Act–Observe–Reflect–Synthesise pattern and uses search, financial-data, and news tools with layered fallbacks. The proposal prioritises free-tier services and defines median time-to-report (TTR) of at most 60 seconds for a resolved NSE/BSE ticker as its primary performance target.

At the prototype stage, the frontend includes a landing page and initial routing, along with agent-activity and tool-indicator components and shared TypeScript types. These provide part of the user-interface foundation. Complete end-to-end execution, report completeness, source grounding, and performance have not been established by the frontend artifacts alone and require integration testing. This report documents the problem, design, current implementation evidence, evaluation plan, limitations, and next steps without presenting unmeasured targets as achieved results.

Keywords: AI agent, company research, tool orchestration, financial information, Go, React, TypeScript.

Contents

1 Introduction **1**
- 1.1 Background and Context . . . 1
  - 1.1.1 Problem Statement . . . 1
- 1.2 Project Objectives . . . 2
- 1.3 Prototype Scope . . . 2

2 Methodology and Design **3**
- 2.1 System Architecture . . . 3
  - 2.1.1 High-Level Design . . . 3
  - 2.1.2 Agent Orchestration . . . 4
  - 2.1.3 Technical Stack . . . 4
- 2.2 Implementation Details . . . 4
  - 2.2.1 Frontend Input and Navigation . . . 4
  - 2.2.2 Agent Activity UI and Shared Types . . . 5
  - 2.2.3 Backend and Data-Source Design . . . 5
  - 2.2.4 Report Synthesis . . . 5
- 2.3 Resilience and Deployment . . . 5

3 Results and Evaluation **6**
- 3.1 Prototype Status . . . 6
- 3.2 Testing Methodology . . . 7
  - 3.2.1 Frontend tests . . . 7
  - 3.2.2 Backend and integration tests . . . 7
  - 3.2.3 Data-grounding review . . . 7
- 3.3 Performance Metrics . . . 7
- 3.4 Analysis . . . 8

4 Conclusion **9**
- 4.1 Summary of Work . . . 9
- 4.2 Key Findings . . . 9
- 4.3 Future Work and Recommendations . . . 9
- 4.4 Final Remarks . . . 10

List of Figures

2.1 Proposed ResearchAgent architecture. Integration and runtime behaviour require validation. 3

List of Tables

2.1 Proposed technology stack and responsibilities. 4

3.1 Prototype status and outstanding verification. 6
3.2 Evaluation metrics and evidence status. 8

Chapter 1

Introduction

1.1 Background and Context

Preliminary research on a listed company may involve reviewing financial snapshots, recent news, company and sector context, competitors, and potential risk factors. These materials are distributed across different websites and vary in freshness and format. A user must identify relevant sources, cross-check claims, and organise the information into a consistent brief.

Large language models can assist with synthesis, but a model-only answer may be incomplete or unsupported if it is not grounded in retrieved evidence. A tool-using agent offers a more structured approach: plan research steps, call external tools, inspect their outputs, retry when evidence is insufficient, and generate a report from the gathered context. The engineering challenge is to coordinate tools reliably and communicate uncertainty when data is unavailable.

ResearchAgent targets this problem with a lightweight web application intended for students and individual investors. The immediate engineering objective is a measurable research workflow for one company at a time, rather than novel model research or stock-price prediction.

1.1.1 Problem Statement

The project proposal identifies four pain points:

- **Fragmentation:** financial information, news, and qualitative context live on separate sites and require manual cross-referencing.
- **Cost barriers:** paid data providers and model APIs may be inaccessible to students and retail investors.
- **Inconsistent structure:** manually compiled notes vary in coverage and organisation.
- **Freshness and completeness:** manual snapshots can become stale and omit relevant news or risks.

ResearchAgent aims to reduce this effort by gathering available evidence and organising it into a repeatable brief. It is an informational research aid, not a substitute for professional financial advice.

1.2 Project Objectives

1. Provide a web interface that accepts a company name or stock ticker.
2. Design an agent loop that plans research and dispatches search, financial-data, and news tools.
3. Synthesize retrieved context into six sections: overview, financial snapshot, news, competitive landscape, risk factors, and bull/bear investment summary.
4. Improve resilience through layered fallbacks and graceful handling of empty or unavailable results.
5. Evaluate TTR, ticker-resolution accuracy, report completeness, fallback frequency, and endpoint availability.
6. Keep deployment practical through a decoupled React frontend and Go API, targeting free-tier hosting and no required managed database.

1.3 Prototype Scope

The repository work documented for this report includes a frontend landing page, initial routing, reusable agent-activity and tool-indicator components, and shared TypeScript types. The proposal describes a broader target system including backend orchestration, data tools, fallbacks, and report generation. Chapter 3 distinguishes these frontend artifacts from capabilities that still need end-to-end verification.

Chapter 2

Methodology and Design

2.1 System Architecture

The target system separates user interaction from research orchestration. The frontend collects a company or ticker and presents activity and the eventual report. The Go API coordinates the request, invokes the model and data tools, and returns structured output. External sources provide search results, financial data, and news.

2.1.1 High-Level Design

Figure 2.1 shows the intended architecture. It is a design view, not proof that every component is integrated and tested.

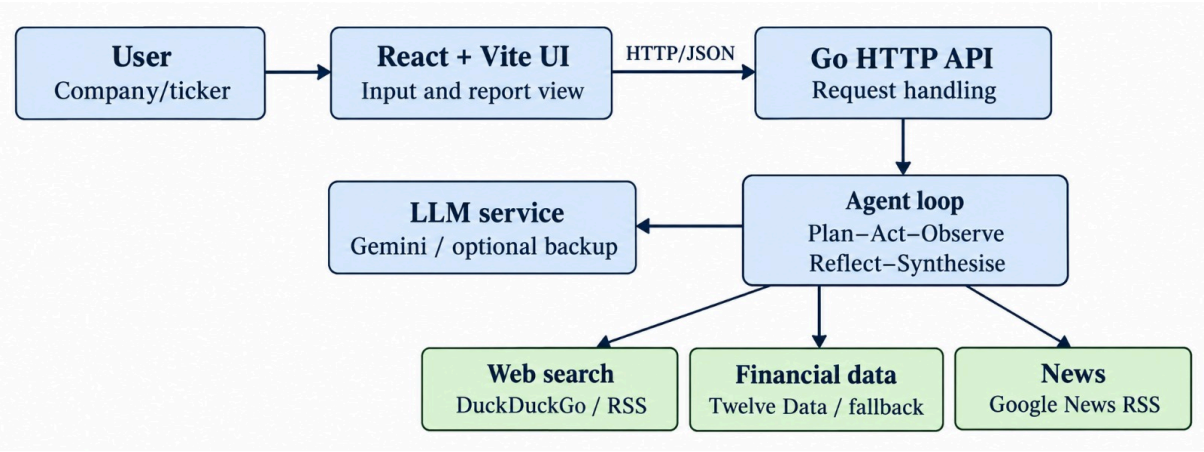


Figure 2.1: Proposed ResearchAgent architecture. Integration and runtime behaviour require validation.

The intended request flow is: (1) accept a company or ticker; (2) resolve or validate the ticker; (3) plan research steps; (4) dispatch tools and inspect their results; (5) retry or use

fallbacks when evidence is insufficient; and (6) synthesise a structured report for display. The implementation should retain source provenance and avoid claiming completeness when sources fail.

2.1.2 Agent Orchestration

The proposal specifies a Plan–Act–Observe–Reflect–Synthesise loop. Planning identifies the information needed. Acting dispatches search, financial-data, or news calls. Observation records tool output. Reflection checks whether the evidence is sufficient and whether a retry or fallback is warranted. Synthesis organises the retrieved context into the final brief. Explicit tool boundaries allow source failures to be handled without treating missing data as facts.

2.1.3 Technical Stack

Technology	**Intended role**
Go standard library	HTTP API and backend orchestration.
React, TypeScript, Vite	Web interface, typed components, and frontend build.
Tailwind CSS	Responsive, dark-mode-first styling.
Gemini API	Primary language-model service for planning and synthesis.
OpenRouter	Optional model-access fallback, subject to configuration.
Twelve Data	Optional primary financial-data API.
Yahoo Finance and Screener.in	Layered fallback sources; availability is not guaranteed.
DuckDuckGo HTML search and Google News RSS	Search and news retrieval with fallback behaviour.
Render	Intended hosting for API and static frontend.
Git/GitHub	Version control and collaboration.

Table 2.1: Proposed technology stack and responsibilities.

2.2 Implementation Details

2.2.1 Frontend Input and Navigation

The frontend is located in `code/analyst-agent-web`. The prototype increment includes a dedicated home page at `src/pages/Home.tsx`, initial application routing in `src/App.tsx`, and styling and configuration updates. These establish the entry point for a company/ticker

query and the initial navigation structure. A route in the frontend does not itself prove that a report page or complete API round trip is operational.

2.2.2 Agent Activity UI and Shared Types

The frontend increment adds `src/components/AgentThinking.tsx` and `src/components/ToolCallBad` and extends `src/types/index.ts`. These components are intended to represent agent progress and tool activity, while shared types provide consistent structures for agent steps, tool calls, messages, and report data. Their presence establishes presentation primitives; live event streaming and backend integration must be tested separately.

2.2.3 Backend and Data-Source Design

The proposal defines a Go HTTP API with an agent route and dedicated search, financial-data, and news tools. Twelve Data is the optional primary financial source, with Yahoo Finance and Screener.in described as fallback sources. Search is intended to try DuckDuckGo and use a Google News RSS fallback when appropriate; Google News RSS also supplies dedicated news retrieval. Public and unofficial sources may change, throttle requests, or become unavailable. Timeouts, bounded retries, and clear error handling are therefore important. The degree of implementation of each backend module should be confirmed against source code and passing tests before being claimed as complete.

2.2.4 Report Synthesis

The intended six sections are: (1) company overview, (2) financial snapshot, (3) recent news, (4) competitive landscape, (5) risk factors, and (6) bull/bear investment summary. Synthesis should be grounded in retrieved tool output. Missing information should be explicitly marked rather than filled with invented values, and the brief should be labelled informational, not financial advice.

2.3 Resilience and Deployment

Risks include source instability, rate limits, model/API quotas, incorrect ticker resolution, and unsupported generated claims. The proposal's mitigations include layered fallbacks, graceful degradation, retries when evidence is insufficient, and source-grounded synthesis. The intended deployment separates the stateless Go API from the static frontend, enabling independent build and deployment. CI is planned to build and vet Go code and type-check/build the frontend; pipeline status should only be reported after verification.

Chapter 3

Results and Evaluation

3.1 Prototype Status

Table 3.1 separates documented frontend work from end-to-end capabilities that require further verification.

Capability	**Evidence / acceptance check**	**Status**
Landing page	Home page source under `src/pages/Home.tsx`.	Frontend implemented
Initial routing	Route setup in `src/App.tsx`.	Frontend implemented
Agent activity UI	`AgentThinking.tsx`.	Component added; live integration unverified
Tool indicator	`ToolCallBadge.tsx`.	Component added
Shared agent/report types	`src/types/index.ts`.	Types added
Ticker resolution	Test known company names and NSE/BSE symbols.	Requires test evidence
End-to-end execution	Verify tool calls, synthesis, and returned report.	Requires integration validation
Six-section completeness	Review each section against retrieved sources.	Not measured here
Median TTR	Measure request receipt to synthesis completion.	Not measured here
Endpoint availability	Probe endpoint during a documented window.	Not measured here

Table 3.1: Prototype status and outstanding verification.

3.2 Testing Methodology

The following tests are proposed for the next validation pass; results should be recorded only after execution.

3.2.1 Frontend tests

- Confirm the landing page renders without console errors.
- Test empty input, whitespace, a known ticker, and a company-name query.
- Confirm navigation and validation behaviour for valid and invalid inputs.
- Test agent-activity and tool-indicator components with sample data, including empty steps and failed tool results.

3.2.2 Backend and integration tests

- Verify health and API routes, status codes, and JSON response structure.
- Exercise each tool with valid queries and simulated timeout, rate-limit, and empty-result conditions.
- Verify a complete request through planning, tool dispatch, observation, retry where applicable, synthesis, and rendering.
- Confirm source failures appear as missing evidence rather than fabricated financial facts.

3.2.3 Data-grounding review

For a sample of generated reports, compare financial figures and news claims with the sources returned during the run. Record whether each claim is supported and whether missing data or uncertainty is communicated. A report should not pass solely because all six headings are present.

3.3 Performance Metrics

The proposal defines time-to-report (TTR) as the primary metric, with a target median of at most 60 seconds for a resolved NSE/BSE ticker. Secondary metrics are ticker-resolution accuracy, data completeness, fallback trigger rate, and endpoint availability. These are targets and evaluation criteria, not measured achievements.

Metric	Measurement	Current evidence
Time-to-report	Median time from API receipt to synthesis over resolved tickers.	Target ≤ 60 s; not measured
Ticker-resolution accuracy	Correct mappings divided by tested company-name inputs.	Baseline required
Data completeness	Reports with six non-empty, source-grounded sections divided by attempts.	Not measured
Fallback trigger rate	Runs invoking fallback divided by total runs, by source.	Not measured
Availability	Successful endpoint probes divided by total probes over a stated window.	Not measured

Table 3.2: Evaluation metrics and evidence status.

The proposal recommends a pilot of 20–30 companies spanning large-cap, mid-cap, and small-cap NSE/BSE listings. A test log should record input, resolved ticker, timestamps, tool and fallback events, section completeness, and manual source-grounding results. Median latency should be reported with sample size and failures; a single successful run cannot establish the target.

3.4 Analysis

The frontend work establishes an entry point and reusable presentation components. These are necessary building blocks but do not establish that the complete pipeline meets performance or quality targets. The next milestone is a reproducible end-to-end request demonstrating ticker handling, tool invocation, source-grounded synthesis, and report rendering. Source fallback may improve resilience while increasing latency, so this trade-off should be measured. Report quality must be evaluated separately from response speed.

Chapter 4

Conclusion

4.1 Summary of Work

ResearchAgent aims to reduce manual effort in preliminary company research by coordinating public information retrieval and model synthesis into a structured investment brief. Its proposed architecture combines a React/TypeScript frontend, Go API, tool-using agent loop, and layered search, financial-data, and news sources. The documented frontend prototype includes a landing page, initial routing, agent-activity and tool-indicator components, and shared types. End-to-end correctness, report completeness, source grounding, and TTR remain to be established through testing.

4.2 Key Findings

- Separating frontend interaction from backend orchestration provides a clear system boundary.
- Agent-activity components and shared types establish a foundation for representing tool use in the interface.
- Source fallbacks and explicit handling of missing evidence are central when relying on public data and free-tier services.
- Credible evaluation requires both operational metrics and factual-grounding review; speed alone is insufficient.

4.3 Future Work and Recommendations

1. Complete and verify the end-to-end API path from ticker input to rendered report.
2. Implement or validate ticker resolution and all three research tools, including fallbacks.

3. Connect live agent-step events to the frontend and test success, retry, empty, and error states.
4. Generate the six-section brief with source references and explicit missing-data handling.
5. Add route-level tests, Go build/vet checks, frontend type checking, and repeatable CI.
6. Run the proposed 20–30-company pilot and report actual TTR, resolution accuracy, completeness, fallback rate, and availability.
7. Add bounded retries, timeouts, request pacing, and caching where appropriate.
8. Review privacy, source terms, and deployment configuration; keep API keys out of source control.

4.4 Final Remarks

ResearchAgent applies established engineering practices to an information-access problem: modular architecture, explicit tool orchestration, source fallback, typed interfaces, and measurable evaluation. Its value depends on the quality and traceability of retrieved evidence, not merely on fluent output. The next stage should prioritise a working end-to-end slice and transparent evaluation against the project's stated criteria.

Bibliography

[1] A. Khanduja, G. Sharma, and V. Choudhary, *ResearchAgent: An Autonomous AI Agent for Stock and Company Research*, Project Proposal for UCS503P, Thapar Institute of Engineering and Technology, August 25, 2026.

[2] Meta Open Source, *React Documentation*. Available: https://react.dev/. Accessed October 2026.

[3] Vite Contributors, *Vite Documentation*. Available: https://vite.dev/. Accessed October 2026.

[4] Microsoft, *TypeScript Documentation*. Available: https://www.typescriptlang.org/docs/. Accessed October 2026.

[5] The Go Authors, *Go Documentation*. Available: https://go.dev/doc/. Accessed October 2026.

[6] Google, *Gemini API Documentation*. Available: https://ai.google.dev/gemini-api/docs. Accessed October 2026.

[7] Twelve Data, *API Documentation*. Available: https://twelvedata.com/docs. Accessed October 2026.

[8] Render, *Documentation*. Available: https://render.com/docs. Accessed October 2026.